Sprawozdanie – zajęcia 6

Obsługa danych wejściowych i wyjściowych, automatyczne uruchamianie wersji serwerowej, porównanie czasów CA i MC

Projekt 1 · Metody analizy dyskretnej w złożonych systemach obliczeniowych · 30.10.2026

Autor: _____

1. Aplikacja w skrócie

Aplikacja generuje mikrostrukturę materiału w 2D i 3D dwoma algorytmami: naiwnym rozrostem ziaren automatem komórkowym (CA) i rozrostem ziaren Monte Carlo (MC, model Pottsa). Składa się z dwóch części:

- **Wersja serwerowa** (C#, .NET 10, aplikacja konsolowa) – czyta parametry z pliku `config.json`, liczy mikrostrukturę i zapisuje wyniki do plików. Nie ma interfejsu, więc może działać na serwerze obliczeniowym.
- **Część użytkowa** (przeglądarka: ASP.NET Core, JavaScript, three.js) – formularz parametrów, zlecanie obliczeń, wizualizacja 2D/3D i zestawienie wyników. GUI nie zawiera kodu algorytmów: zapisuje `config.json` i uruchamia wersję serwerową jako osobny proces.

2. Dane wejściowe i wyjściowe

Każde uruchomienie ma własny folder `OutputTXT/runs/run_<data_godzina>/`. Części aplikacji wymieniają dane wyłącznie przez pliki:

Plik	Kierunek	Zawartość
`config.json`	wejście	algorytm (CA/MC), rozmiar siatki X × Y × Z, sąsiedztwo (Moore / von Neumann), warunki brzegowe (periodyczne / pochłaniające), seed; dla CA liczba zarodków; dla MC liczba stanów Q, kroki MCS, energia granicy J i sposób losowania nowego stanu
`Micro.txt`	wyjście	mikrostruktura końcowa: nagłówek z rozmiarem, potem jedna komórka na linię „x y z id”
`StepTimes.txt`	wyjście	przebieg: czas każdego kroku; dla CA liczba zapełnionych i pustych komórek, dla MC energia granic
`Time.txt`	wyjście	podsumowanie: czasy alokacji, stanu początkowego, rozrostu i zapisu pliku, liczba ziaren na końcu
`console.txt`	wyjście	zapis wyjścia konsoli wersji serwerowej (zapisuje go GUI)

Dane wejściowe są sprawdzane dwa razy: w GUI (komunikaty przy formularzu, limit 8 mln komórek) i w wersji serwerowej (błąd w pliku konfiguracyjnym kończy program z opisem i kodem wyjścia 1). Ten sam seed daje identyczną mikrostrukturę, więc każde uruchomienie można powtórzyć z jego `config.json`.

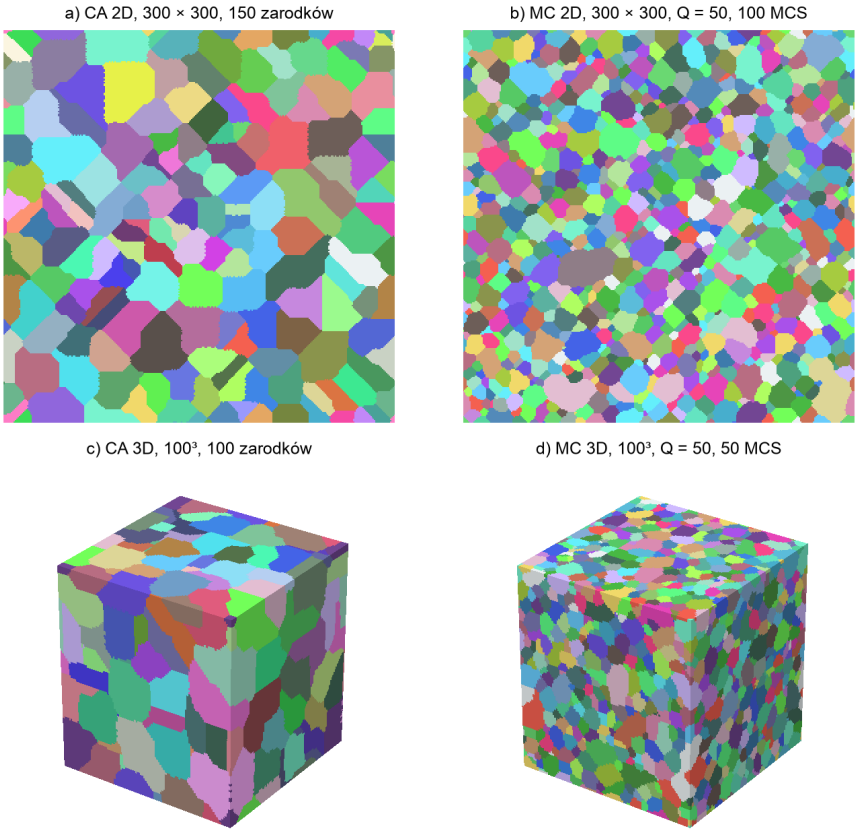
3. Automatyczne uruchamianie wersji serwerowej

Serię zleca się w GUI: parametry bazowe z formularza, wybrane algorytmy, lista rozmiarów n (siatka n^3) i liczba powtórzeń (kolejne seedy). Aplikacja:

1. generuje wszystkie pliki `config.json`, po jednym folderze na zlecenie,
2. wpisuje zlecenia do kolejki i uruchamia wersję serwerową po kolei – uruchomienia nie konkurują o procesor, więc czasy są porównywalne,
3. po zakończeniu zbiera wyniki z plików `Time.txt` w zakładce „Zestawienie wyników” (wykresy i tabela) i udostępnia je jako plik `wyniki.csv`.

Tę samą automatyzację można uruchomić bez GUI: `scripts/run-all.sh configs/*.json` liczy po kolei każdy podany plik konfiguracyjny w osobnym folderze `OutputTXT/batch/<nazwa>/`.

4. Przykładowe mikrostruktury

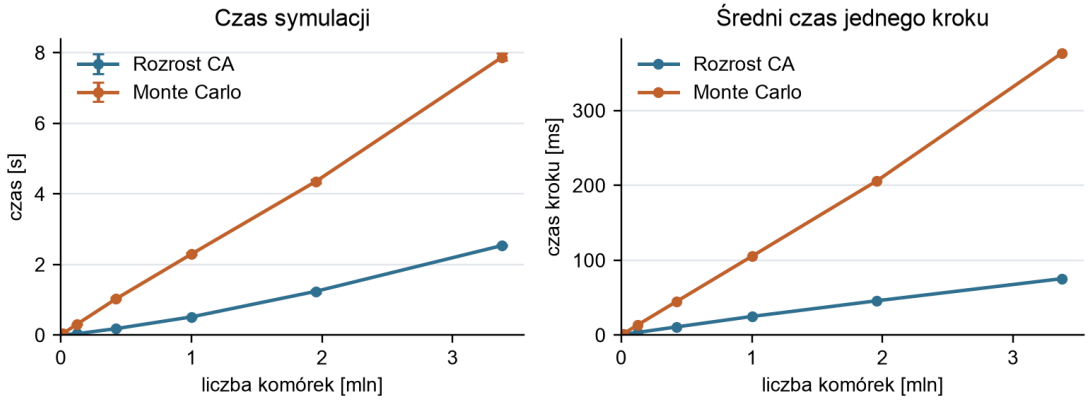


Rys. 1. Przykładowe mikrostruktury: a, b – przekroje 2D; c, d – rzut ścian zewnętrznych kostki 3D. Sąsiedztwo Moore'a, brzegi periodyczne; w MC nowy stan losowany spośród sąsiadów. Kolor oznacza numer ziarna.

CA daje ziarna o prostych, kanciastych granicach – ich kształt narzuca kwadratowa siatka i sąsiedztwo Moore'a. MC daje zaokrąglone, równoosiowe ziarna, bo zmiana stanu jest przyjmowana tylko wtedy, gdy nie zwiększa energii granic, więc granice się skracają.

5. Porównanie czasów CA i MC

Seria zlecona z GUI: 36 uruchomień – CA i MC dla siatek 25³–150³ (od 15 625 do 3 375 000 komórek), po 3 powtórzenia (seed 1–3). Wspólne parametry: sąsiedztwo Moore'a, brzegi periodyczne. CA: 100 zarodków, symulacja do zapełnienia siatki. MC: Q = 50, 20 kroków MCS, nowy stan losowany z całego zbioru. Środowisko: Apple M2 (8 rdzeni), 16 GB RAM, macOS 27, .NET 10, build Release; obie wersje liczą na jednym rdzeniu.



Rys. 2. Czas symulacji (średnia z 3 uruchomień, słupki: minimum i maksimum) oraz średni czas jednego kroku w funkcji liczby komórek.

Siatka	Komórki	CA: kroki	CA: czas	MC: czas (20 MCS)	MC / CA
25^3	15 625	5,3	6,9 ms	42,7 ms	6,2 ×
50^3	125 000	11,0	39,8 ms	312,1 ms	7,8 ×
75^3	421 875	16,7	182,2 ms	1,03 s	5,6 ×
100^3	1 000 000	20,7	515,1 ms	2,30 s	4,5 ×
125^3	1 953 125	27,0	1,24 s	4,35 s	3,5 ×
150^3	3 375 000	33,7	2,53 s	7,86 s	3,1 ×

- **MC jest wolniejszy od CA 3,1–7,8 razy.** Przewaga CA maleje z rozmiarem siatki, bo przy stałej liczbie zarodków CA potrzebuje coraz więcej kroków (5 → 34), a MC zawsze wykonuje 20.
- **Czas MC rośnie liniowo z liczbą komórek:** 8 razy więcej komórek (50^3 → 100^3) daje 7,4 razy dłuższy czas.
- **Jeden krok MC jest ok. 4,2 razy droższy od kroku CA** (100^3: 105,3 ms wobec 24,9 ms). MC w każdym kroku rozpatruje każdą komórkę na granicy i tasuje kolejność wszystkich komórek, a CA liczy tylko puste komórki, których z każdym krokiem ubywa.
- **Wyniki są powtarzalne:** od siatki 50^3 rozrzut między powtórzeniami wynosi najwyżej 7% średniej (przy 25^3, gdzie czasy liczą się w milisekundach, do 12%).

Duże siatki. GUI przyjmuje siatki do 200^3 (8 mln komórek), bo większych przeglądarka nie wyświetli płynnie. Większe siatki policzono wersją serwerową z terminala, z tymi samymi parametrami (jeden wątek):

Siatka	CA: kroki	CA: symulacja	MC (20 MCS): symulacja	RAM (szczyt)	Plik Micro.txt
200^3	45	7,7 s	35 s	133 MB	106 MB
250^3	56	19 s	84 s	198 MB	212 MB
300^3	72	39 s	157 s	282 MB	373 MB
350^3	77	71 s	ok. 4 min (szacunek)	426 MB	599 MB

Czasu MC dla 350^3 nie mierzono – oszacowano go z czasu na komórkę na krok dla 300^3 (ok. 290 ns). Ogranicza czas, nie pamięć. Rozsądne maksimum na jednym wątku (od pół minuty do minuty) to **CA 300^3 i MC 200^3** – te rozmiary posłużą do porównania z wersją wielowątkową w drugiej części projektu. CA ma stały koszt ok. 21 ns na komórkę na krok, a MC od 200^3 zwalnia na komórkę ok. 2,5 razy (115 → 290 ns). Najpewniej dlatego, że dane przestają mieścić się w pamięci podręcznej procesora, a MC odwiedza komórki w losowej kolejności.

Ciekawostka: sama zmiana wersji .NET. W trakcie projektu środowisko zaktualizowano z .NET 7 do .NET 10. Kod i mikrostruktury się nie zmieniły, a obliczenia przyspieszyły dzięki lepszemu kompilatorowi JIT (100^3, średnia z 3 uruchomień):

Pomiar	.NET 7	.NET 10	Zmiana
CA – symulacja	799 ms	505 ms	−37%
MC (20 MCS) – symulacja	2630 ms	2250 ms	−14%
CA, siatka jako wektor – benchmark	433 ms	277 ms	−36%

6. Wnioski

- Wersja serwerowa i GUI są rozdzielone i wymieniają dane tylko przez pliki, więc obliczenia można zlecać z GUI, ze skryptu albo ręcznie.
- Seria 36 uruchomień – od wygenerowania plików konfiguracyjnych do zebrania wyników na wykresie – trwała ok. 1 minuty i nie wymagała obsługi.
- CA jest szybszy od MC, ale MC daje bardziej realistyczne, zaokrąglone ziarna.
- Najtańszą optymalizacją okazała się aktualizacja .NET 7 → .NET 10: do 37% krótszy czas bez zmiany ani jednej linii kodu.
- Obie wersje liczą na jednym rdzeniu z ośmiu; w drugiej części projektu porównamy je z wersją wielowątkową (CA 300^3, MC 200^3).